Same recipe, different model: cross-system reproducibility of byte-level language-model pretraining holds at 10M but fails at 50M parameters

Anonymous authors
Paper under double-blind review

Abstract

Neural-network training is expected to be reproducible: the same recipe—identical seed, data, and code—should yield the same model. In practice, results differ across hardware, but the phenomenon is poorly quantified at scale because repeating large-model pretraining many times is prohibitively expensive. We pretrained byte-level Transformer language models (10M–124M parameters) on a 300 MB permissively licensed (CC-BY/CC0) neuroscience open-access corpus, on a zero-cost heterogeneous fleet spanning an NVIDIA CUDA GPU, an Apple-Silicon Metal (MPS) backend, and CPUs across two machines, sweeping seeds, model sizes, and numerical precisions for 4,000 steps. Hypotheses, thresholds, and an anti-rescue ledger were cryptographically sealed before any run; cross-backend divergence was measured as held-out next-token prediction disagreement, with same-seed replicates as determinism controls. Within a machine and backend, CPU and CUDA reproduced bit-identically while MPS was self-non-reproducible (0.07%). Across systems, reproducibility was size-dependent over the two sizes compared: 10M models agreed (0.2% prediction disagreement) whereas 50M models diverged into different models ($13.2\% \pm 1.0\%$, three seeds) despite near-identical validation loss ($|\Delta| = 0.012$)—an "equally-good-but-different" outcome that aggregate metrics hide. In single-seed probes, lower precision brought divergence onset earlier (fp16 by $\leq$ step 50, bf16 by step 400). Our pre-registered persistence hypothesis was refuted at 10M; the resulting scale-dependence is reported as a post-hoc, hypothesis-generating observation, not a confirmed law. The single divergent point confounds backend with machine and build, so we attribute it to a cross-system difference rather than the numerical path alone. Small-model, many-run studies on open corpora make this failure mode measurable and reproducible.

1 Introduction

Reproducibility is a load-bearing assumption of computational science: an independent re-execution of the same procedure should reproduce the same result. For neural-network training this assumption is routinely invoked when a "model" is treated as a deterministic function of its recipe — the training data, the random seed, the code, and the hyperparameters. In practice, however, the mapping from recipe to trained weights also passes through a hardware- and library-specific numerical path, and that path is not bit-neutral. Floating-point addition is not associative, reduction orders differ between CPU, GPU, and accelerator kernels, and several high-performance kernels are non-deterministic by default (Goldberg, 1991; PyTorch Team; NVIDIA). The practical question is not whether such differences exist but whether they matter — whether they stay in the numerical noise floor or grow into behaviorally different models.

This question is under-measured precisely where it matters most. Characterizing run-to-run and cross-hardware variability requires training the same configuration many times, but at the scale of contemporary language models a single pretraining run can cost thousands of GPU-hours, so repetition is rare and variability is usually reported, if at all, as a single seed's loss curve (Bouthillier et al., 2021; Kaplan et al., 2020). The measurement window that *is* affordable — small models trained many times, across heterogeneous hardware

— has been largely unused. Small models are not merely cheap stand-ins; because the same numerical non-associativity operates at every scale, they let us ask a sharper question: *as a model grows, does cross-hardware divergence stay negligible, or does it emerge?*

We approach this with three deliberate design choices. First, the study is **pre-registered**: the hypotheses, decision thresholds, feasibility caveats, and an explicit anti-rescue ledger were written and cryptographically sealed before any full run, so that a surprising result cannot be retrofitted into a confirmed prediction. Second, it is **zero-cost**: all training runs on a small heterogeneous fleet already on hand — an NVIDIA consumer GPU (CUDA), an Apple-Silicon machine (Metal/MPS), and CPUs on two different microarchitectures — so the heterogeneity that is usually a nuisance becomes the independent variable. Third, it is demonstrated on an **open, permissively licensed domain corpus**: roughly 300 MB of CC-BY/CC0 neuroscience open-access full text, so that both the corpus and the resulting artifacts can be released for exact re-derivation.

Our contribution is threefold. (i) With pre-registered thresholds and three-seed error bars, we measure cross-system divergence in byte-level language-model pretraining at two model sizes and find it **size-dependent over that range**: negligible at 10M parameters (0.2% prediction disagreement) but substantial at 50M (13.2%). We report this as a two-point contrast; whether it reflects a monotone *emergence with scale* is a hypothesis it motivates but that two sizes cannot confirm. (ii) We show this divergence is **hidden by aggregate metrics** — models disagreeing on 13% of held-out next-token predictions can have near-identical validation loss — and that, in single-seed probes, it is **worsened by low precision**, with bf16 and fp16 bringing onset hundreds to thousands of steps earlier. (iii) We provide a **reproducibility apparatus** — a sealed pre-registration with an anti-rescue ledger, a determinism-controlled harness, an openly licensed corpus with recorded provenance, and per-run environment fingerprints — that makes the phenomenon measurable on commodity hardware and directly re-runnable by others.

2 Methods

2.1 Corpus

We built a domain corpus from the NCBI PubMed Central (PMC) Open Access subset via the E-utilities API (`esearch`/`efetch`), restricted to neuroscience full text. Because the `open access[filter]` still admits non-commercial and no-derivatives licenses, every article was gated on its JATS `<license>` element and **only permissive licenses were kept — CC0, CC-BY, and CC-BY-SA** (CC-BY-SA was permitted by the gate but none appeared in the final corpus) — with all rejections logged for auditability. Article bodies were extracted from the JATS XML (tables, figures, reference lists, and mathematics removed), concatenated in ascending PMCID order with a document separator, and stored as a raw byte stream. The final corpus is 300.12 MB (6,961 documents: 6,933 CC-BY, 28 CC0; zero NC/ND contamination) with recorded SHA-256, split 99.5/0.5 into train and validation. The collector is stdlib-only and rate-limited to NCBI's guidance ($\leq$3 requests/s). Corpus assembly is deterministic, so the byte stream is reproducible from the document set.

2.2 Model and training harness

Models are byte-level (vocabulary 256) decoder-only Transformers. Three size presets were used; the preset names (10M/50M/124M) are nominal size-class labels, and the actual parameter counts are given below (all hyperparameters are stated for exact reproducibility):

preset (nominal)	d_model	layers	heads	block (context)	actual params
10M	384	6	6	256	10.9M
50M	640	10	10	256	49.7M
124M	768	16	12	512	114.2M

(The "124M" label follows the GPT-2 naming convention but is not an exact GPT-2 configuration; our preset has 114.2M parameters. All comparisons use the actual models above.) Training used AdamW (lr

3e-4), batch 16, gradient clipping 1.0, and a fixed data-sampling order independent of the model seed, for 4,000 steps. The harness records, at eight milestone steps (50, 100, 200, 400, 800, 1,600, 3,200, 4,000), the validation loss, the argmax next-token predictions on a fixed 4,096-token held-out probe, a SHA-256 fingerprint of all weights, and per-parameter statistics. Every run's output JSON records the exact PyTorch version, so any accidental interpreter mismatch is self-evident.

Determinism was requested via `torch.use_deterministic_algorithms`, `cudnn.deterministic`, and `CUBLAS_WORKSPACE_CONFIG` (set before the framework import). We note that CUDA memory-efficient attention warns that it is non-deterministic; we therefore treated within-machine, within-backend bit-identity as an empirical control to be verified rather than assumed (see Results).

2.3 Heterogeneous fleet

All computation used hardware already on hand at no marginal cost: an NVIDIA GeForce RTX 3060 (CUDA 12.6, driver 560.94; Windows) providing the CUDA backend and an AMD-based CPU; and an Apple-Silicon Mac mini (macOS) providing the Metal Performance Shaders (MPS) backend and an ARM CPU. Both machines ran the same PyTorch release (2.12.1) but necessarily different platform builds — `2.12.1` (Apple/MPS) versus `2.12.1+cu126` (CUDA) — a difference we treat as part of the cross-system confound rather than claim away as "identical." Feasibility was hardware-bounded and reported honestly: the 124M model fit only on CUDA (MPS exhausted 16 GB unified memory; CPU was too slow for 4,000 steps), so the cross-backend comparison spans 10M and 50M, with 124M contributing within-backend controls.

2.4 Pre-registration and anti-rescue

Before any full run, we sealed a pre-registration document (hypotheses H1–H4, decision bands, feasibility caveats, and a seven-item anti-rescue ledger) by committing it with its own SHA-256 recorded in the commit message, establishing a time-gate. The ledger explicitly forbade, among other moves, shifting thresholds to fit results, dropping disagreeing seeds, extending training until divergence appeared, and re-scoping hypotheses post hoc. All verdicts below are reported against the sealed thresholds without modification.

2.5 Divergence metric and hypotheses

Cross-backend divergence between two runs is the percentage of the 4,096 fixed-probe positions at which their argmax next-token predictions differ, at a given step; the final-step value is the primary outcome and is averaged over seeds {0, 1, 2}. The pre-registered hypotheses were: **H1 (persistence)** — CUDA↔CPU final disagreement $\geq$10% at both 10M and 50M (three-band verdict: $\geq$10% supported / 5–10% diminished-but-survives / $<$5% washed out); **H2 (onset)** — the step at which divergence exceeds 1% moves earlier with model size; **H3 (precision)** — bf16/fp16 reach onset earlier than fp32; and **H4 (loss masking)** — models differing by $\geq$10% in predictions differ by $<$0.05 in validation loss. Because 50M has no feasible CPU run, its cross-backend pair is CUDA↔MPS, a substitution recorded at seal time. The determinism control required same-machine, same-backend, same-seed replicates to be bit-identical (with MPS self-non-reproducibility a documented expected exception).

3 Results

We trained 27 models to 4,000 steps each (Mac: 11; NVIDIA host: 16), all recording the expected PyTorch build and full eight-point milestone curves. Results are reported against the sealed thresholds. Unless stated otherwise, cross-backend disagreement is the three-seed mean; onset and precision traces (H2, H3) are **single-seed (seed 0)** and are labelled as such throughout.

3.1 Determinism controls hold; MPS is the exception

Same-machine, same-backend, same-seed replicates were **bit-identical** on both the CPU and the CUDA backends (weight SHA-256 match; 0.00% prediction disagreement at step 4,000). We confirmed this empirically rather than assuming it: despite the CUDA memory-efficient-attention non-determinism warning,

two seed-0 CUDA runs produced identical weight fingerprints, so deterministic kernels were **achieved**, not merely requested. The **Metal (MPS) backend was self-non-reproducible**: two identical-seed runs disagreed on 0.07% of predictions with different weight fingerprints, matching the pre-registered expectation. Cross-backend differences below are therefore read against a CPU/CUDA baseline that is itself exactly reproducible.

3.2 H1: reproducible at 10M, divergent at 50M (persistence hypothesis refuted)

At **10M parameters**, all six cross-backend/cross-machine pairs agreed almost completely: CUDA↔CPU disagreement was **0.2%** (three seeds), and every other pair (CPU↔CPU across the two machines, CUDA↔MPS, MPS↔CPU) fell between 0.1% and 0.2% — in the pre-registered **washed-out** band (<5%). At **50M parameters**, the feasible cross-backend pair, **CUDA↔MPS**, disagreed on **13.2% ± 1.0%** of predictions (mean ± SD over seeds {0,1,2}) — in the **supported** band (≥10%) (Fig. 2). The sealed **H1**, which required ≥10% at *both* sizes, is therefore **refuted as stated**. The literal finding is that hardware reproducibility is **size-dependent over the two sizes tested**: reproducible at 10M, divergent at 50M. We treat the broader claim that divergence *emerges as a function of scale* as a post-hoc, unregistered observation motivated by this contrast, not as a confirmed law (see Discussion); two sizes cannot fit a scaling curve, and the 50M pair confounds backend with machine and PyTorch platform build.

3.3 H2: onset present at 50M, absent at 10M (single-seed)

Tracking seed-0 disagreement across steps, the 10M CUDA↔CPU pair never crossed the 1% onset threshold (final 0.3%). The 50M CUDA↔MPS pair crossed 1% by **step 800** (9.2%) and first reached the ≥10% band by **step 1,600**; thereafter it fluctuated non-monotonically between about 9% and 12% (12.3 → 10.8 → 11.9% at steps 1,600/3,200/4,000) rather than climbing smoothly (Fig. 2). With only one size exhibiting any divergence and a single seed, we cannot fit an onset-versus-size relationship; the direction (larger diverges, smaller does not) is merely consistent with the pre-registered H2.

3.4 H3: lower precision brings divergence earlier (supported, single-seed)

Holding the CUDA backend and seed (0) fixed at 50M, we compared fp32 against reduced precision (Fig. 3). Divergence from the fp32 reference appeared earlier and larger at lower precision: **fp32↔bf16** crossed the 1% onset by **step 400**, and **fp32↔fp16** already disagreed on 6.4% of predictions by **step 50** — i.e. its onset is at or before our first milestone (≤ step 50), below the resolution of the grid. The fp16 trace is itself strongly non-monotone (51.8 → 45.1 → 25.7 → 71.9% at steps 400/800/1,600/4,000); we therefore report the direction (low precision → earlier, larger divergence) as supporting **H3** but treat the specific endpoint magnitudes, including the 71.9% value, as single-seed and not robust. Additional seeds for the precision axis are pre-registered future work.

3.5 H4: aggregate loss masks the divergence (supported)

For the 50M CUDA↔MPS pair that disagreed on 13.2% of predictions, the corresponding validation-loss difference was only $|\Delta| = \mathbf{0.012}$ — below the pre-registered 0.05 masking threshold. The two backends produced models that are **equally good by validation loss yet behaviorally different**, supporting **H4**: a scalar aggregate metric conceals the cross-system divergence that a prediction-level comparison reveals. This corroborates, on a CPU/CUDA/Metal backend set and an open corpus, the same-accuracy/different-prediction effect reported for GPU architectures by prior verifiable-training work (Srivastava et al., 2024); we do not claim it as novel (see Discussion).

3.6 Summary

Within a machine and backend, CPU and CUDA training is exactly reproducible and MPS is not; across systems, models are reproducible at 10M but diverge at 50M despite matched loss; and, in single-seed probes, reduced precision moves divergence earlier. Of the four pre-registered hypotheses, H3 and H4 were supported,

H2 was directionally consistent, and H1 was refuted. We report the refutation as a refutation; the scale-dependence it exposes is developed in the Discussion as a hypothesis for future, adequately-powered work, not as an established result.

4 Discussion

Our literal result is narrow and solid: on the same 300 MB neuroscience corpus, the same training recipe produces essentially one model at 10M parameters across CPU, CUDA, and Metal (0.2% prediction disagreement) but two different models at 50M (13.2%) that share the same validation loss. Two design choices make even this narrow result unusually credible. Because the study was pre-registered and sealed, we could not convert a surprising outcome into a claimed prediction: our persistence hypothesis (H1) was refuted at 10M, and the anti-rescue ledger prohibited moving the threshold or excising the small-model result. And because within-machine CPU/CUDA training was *verified* bit-identical, the 50M divergence is not generic run-to-run noise.

What we do not claim. We are careful to separate the registered result from the story it suggests. First, "divergence emerges with model scale" is a claim about a *function* of size; we have two sizes, so we treat scale-emergence as a **post-hoc, unregistered hypothesis** that this contrast motivates, not as a demonstrated law. Second, the single divergent point (50M, CUDA↔MPS) confounds backend, machine, operating system, and PyTorch platform build (2.12.1 vs 2.12.1+cu126); the determinism controls exclude run-to-run noise but not these system axes. We therefore attribute the 50M divergence to a **difference between two compute systems**, not to the numerical/hardware path *per se* — isolating the latter requires a same-machine cross-backend comparison at 50M, which CPU throughput made infeasible here and which is the first item of future work.

Non-monotonicity, and the limits of the regime story. Across our full size range the divergence is not monotone: an overfitting toy pilot (0.8M-parameter character-level Shakespeare) diverged by 17–19%, our 10M real-corpus model by 0.2%, and 50M by 13.2% (18% → 0.2% → 13.2%). A tempting reconciliation is a *regime* argument — sharp loss surfaces in the memorization regime amplify tiny numerical differences, smoother generalization surfaces do not, and added capacity sharpens the surface again. We find this mechanism plausible but we offer **no direct evidence** for it (no curvature, gradient-noise, or overfitting-gap measurement), so we do not assert it. The honest present statement is that divergence is **non-monotone across scale and its mechanism is unresolved**; distinguishing a genuine scale effect from a regime (overfitting) effect, by instrumenting loss-surface sharpness across matched regimes, is a concrete next study.

Practical implication, appropriately bounded. The combination of H1 and H4 carries the actionable message. On the two sizes tested, a developer who validates a pipeline by comparing final loss across machines would see agreement while the resulting models differ on more than one in ten predictions. Whether this masked divergence continues to grow at larger scale is untested and should not be assumed from two points; but it does not shrink between our two sizes, and single-seed probes suggest reduced precision (now standard) brings its onset dramatically earlier. Prudence therefore favors verifying training-pipeline portability at the level of predictions, not aggregate loss, and pinning the numerical/system path when exact reproducibility is required.

Positioning (see also Related work). That floating-point non-associativity and non-deterministic kernels make training hardware-dependent is known [goldberg1991; shanmugavelu2024fpna; zhuang2022; summers2021], and — most relevant to our H4 — prior work on verifiable training already reports that the same recipe on different GPU architectures can yield similar aggregate accuracy but substantially different predictions (Srivastava et al., 2024). We therefore do **not** claim the same-loss/different-predictions phenomenon as novel; our H4 corroborates it on a different backend set (CPU/CUDA/Metal rather than GPU–GPU) and an open corpus. Our specific, narrower delta is a **pre-registered, determinism-controlled, prediction-level** measurement of how cross-system divergence depends on **model size** for **training from scratch** — an axis these works do not isolate, and one where inference-side evidence points the *other* way (token-probability nondeterminism reported size-independent from 270M to 12B (Fu et al., 2026)). That contrast is itself a reason the training-scale question is worth posing; we pose it and give a two-point first probe, not a set-

tled trend. The claim is the disciplined, scale-resolved measurement, not the existence of hardware-induced divergence.

Limitations. (i) Two sizes contribute to the cross-system comparison; 124M fit only on CUDA. (ii) The 50M pair is singly confounded (above). (iii) H2 and H3 rest on a single seed. (iv) The regime mechanism is unmeasured. (v) Onset is resolved only at eight milestone steps. None threatens the registered qualitative result — reproducible at 10M, divergent at 50M with matched loss — but each bounds how far it can be generalized.

Outlook. Beyond closing these gaps (a second backend at 124M via quantization or gradient accumulation; a same-machine 50M cross-backend pair; multi-seed precision runs; a loss-surface probe), the apparatus itself is the durable contribution: a sealed pre-registration, a determinism-controlled harness, an openly licensed corpus with recorded provenance, and per-run environment fingerprints, all runnable on commodity hardware at no cost — an instrument the community can use to map where pretraining stops being reproducible.

5 Related work and positioning

paragraph. Written after a literature search; the honest delta is narrower than an early draft implied — the *phenomenon* is known, our contribution is a pre-registered, scale-resolved, open-corpus measurement of it.)

Floating-point non-associativity and numerical reproducibility. That summation order changes floating-point results is textbook (Goldberg, 1991), and its consequences for HPC and deep-learning reproducibility have been characterized directly (Shanmugavelu et al., 2024). Parallel GPU reductions, kernel non-invariance, and batch-size effects are established implementation-level sources of run-to-run and cross-device variance (Shanmugavelu et al., 2024; PyTorch Team; NVIDIA). We do not re-establish these; we take them as the mechanism and ask a downstream question.

Hardware nondeterminism yields behaviorally different models at matched aggregate performance. Most directly related to our H4, prior work on verifiable training shows that the *same* recipe on *different GPU architectures* can produce models with similar aggregate accuracy yet substantially different predictions, and treats this as an obstacle to be controlled (Srivastava et al., 2024). Related analyses characterize numerical sources of nondeterminism in LLM inference at temperature zero (Yuan et al., 2025), and optimization-level work documents instability and nondeterminism during training (Summers & Dinneen, 2021; Zhuang et al., 2022). Our H4 result — 13.2% prediction disagreement at near-identical validation loss — therefore **corroborates**, on a different backend set (CPU/CUDA/Metal rather than GPU–GPU) and an open corpus, a phenomenon these works already report; we do not claim it as novel.

Does it scale, and does training differ from inference? The axis we probe — how divergence depends on model size — has been examined for *inference*: measuring token-probability nondeterminism across Gemma-family models from 270M to 12B, prior work finds the magnitude of nondeterminism largely *independent* of model size (Fu et al., 2026). Our (weaker, two-point) result points the other way for *training from scratch* — reproducible at 10M, divergent at 50M — which we read not as a contradiction but as an open question: inference nondeterminism and training-from-scratch divergence are different quantities (the latter compounds over optimization steps), and whether training divergence is genuinely size-dependent is, on our data, a first probe rather than a settled trend. Cross-backend reproducibility has also been attacked at the *deployment* level, with configuration-first frameworks that detect and mitigate numerical drift when a fixed model is run on CPU/GPU/compiled runtimes (Li, 2025); that setting (a trained model, moved across backends) is distinct from ours (a model *trained* on each backend). Training-time reproducibility itself has an established taxonomy of software and hardware nondeterminism sources and mitigations (Chen et al., 2022), on which we build.

Variance and reproducibility methodology in ML. A methodological literature argues for treating training variance as a first-class quantity and for reporting it across seeds and tooling (Bouthillier et al., 2021; Henderson et al., 2018; Pineau et al., 2021). We adopt that stance and add pre-registration with an anti-rescue ledger, which is uncommon in this systems-ML setting.

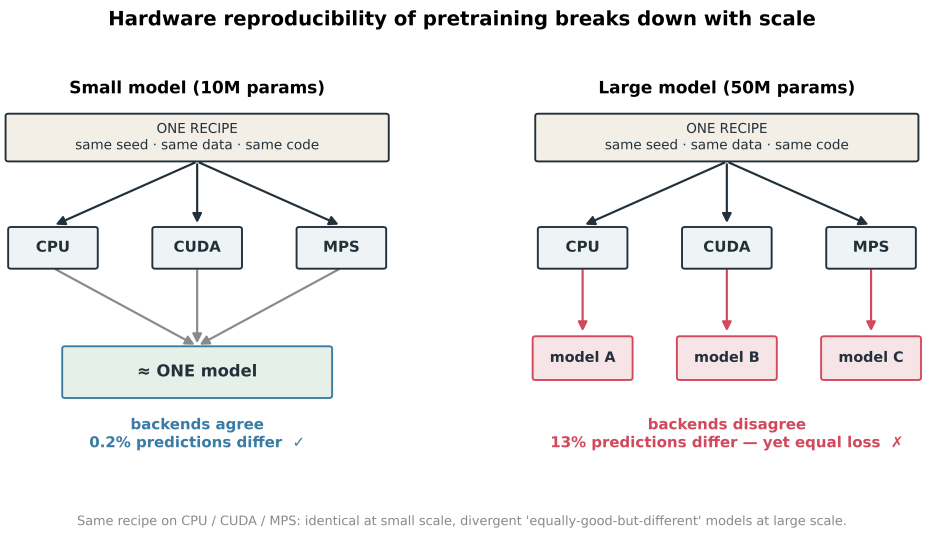

Figure 1: Graphical abstract. One training recipe (same seed, data, code) on CPU/CUDA/MPS yields a near-identical model at 10M (0.2% prediction disagreement) but divergent "equally-good-but-different" models at 50M (13.2%, equal loss).

Precision. Reduced-precision training is known to be less numerically stable, with fp32 near-deterministic and bf16/fp16 more variable (Shanmugavelu et al., 2024; Yuan et al., 2025); our single-seed H3 probe is consistent in direction (lower precision → earlier onset) and is offered as corroboration rather than a controlled precision study.

Our delta (one sentence per axis). Relative to the above: (i) we give a first probe of how cross-system divergence in **training from scratch** depends on **model size** — a question left open by inference-side results that find nondeterminism size-independent (Fu et al., 2026) — finding it washed out at 10M and material at 50M; (ii) we do so under a **sealed pre-registration** with an anti-rescue ledger, so a refuted hypothesis is reported as refuted; (iii) we compare at the level of **held-out predictions** with same-seed **bit-identity controls**, not aggregate loss alone; and (iv) we release an **openly licensed corpus** and per-run environment provenance so the measurement is exactly re-runnable on commodity hardware. The novelty is the disciplined, scale-resolved *instrument and measurement*, not the existence of hardware-induced divergence.

Data and code availability

All code, the sealed pre-registration, the analysis, and the figures are available at `https://github.com/tckamijo/repro-pretrain` (archived at Zenodo, DOI: `10.5281/zenodo.21428182`). The corpus is not redistributed as raw text; it is exactly reconstructible from the PMC Open Access subset using the included collector and verifiable against the recorded SHA-256 (`9d6b168e...5b0a27`, 300,120,401 bytes). Per-run outputs (validation loss, held-out predictions, weight fingerprints, PyTorch build) are provided for all 27 runs, and each figure ships with a provenance sidecar. A claim-to-source verification worksheet is included.

Competing interests

The author declares no competing interests.

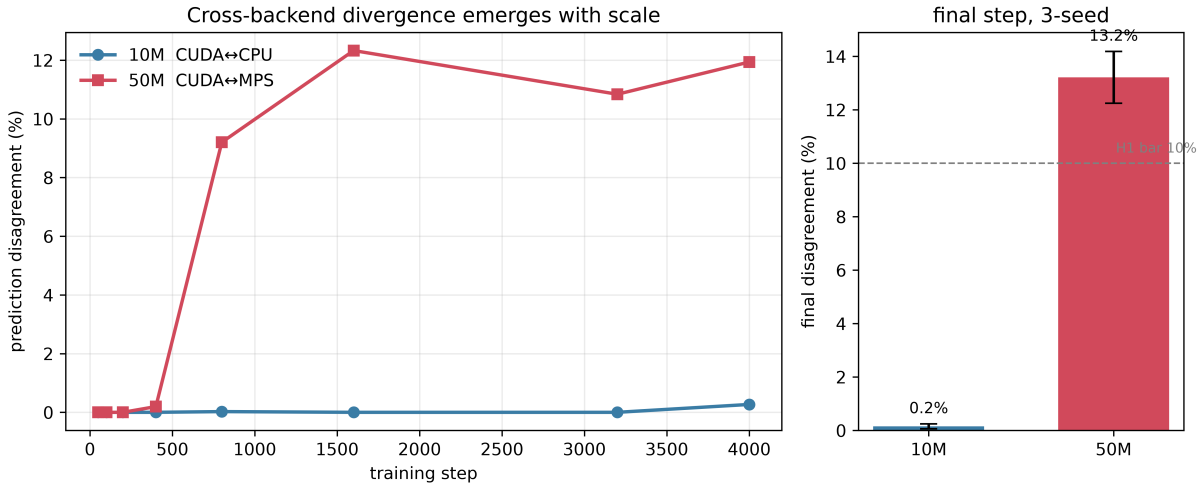

Figure 2: Cross-system prediction disagreement. *Left:* vs. training step (seed 0) for 10M CUDA↔CPU and 50M CUDA↔MPS. *Right:* final-step disagreement by size; bars are the mean over seeds $\{0, 1, 2\}$, error bars ± 1 SD; dashed line marks the pre-registered 10% band.

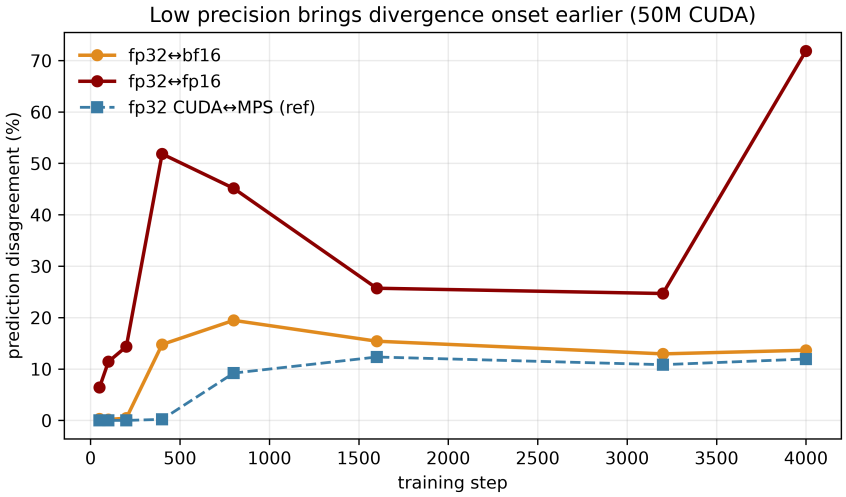

Figure 3: Precision and divergence onset at 50M on CUDA (single seed): fp32↔bf16 and fp32↔fp16, with fp32 CUDA↔MPS for reference. Onset $\leq$ step 50 (fp16) and step 400 (bf16).

Statement on the use of AI tools

In preparing this work the author used a large language model (Anthropic's Claude) as an assistive tool: drafting and editing the manuscript; writing and scaffolding the data-collection, training-harness, analysis, and figure-generation code; assisting with the literature search; and running an internal adversarial review. All experiments were executed by the author on the author's own hardware, and every reported quantitative result was produced by the analysis code and independently verified by the author against the raw per-run outputs. The author designed the study, sealed the pre-registration, checked all numbers, claims, and citations, and takes full responsibility for the content and its scientific validity. No data, results, or citations were fabricated.

References

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming Jiang. Towards training reproducible deep learning models. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022.

Tairan Fu, Gonzalo Martínez, Javier Conde, Carlos Arriaga, Pedro Reviriego, Xiuyuan Qi, and Shanshan Liu. Beyond reproducibility: Token probabilities expose large language model nondeterminism. *arXiv preprint arXiv:2601.06118*, 2026.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Zehua Li. Toward reproducible cross-backend compatibility for deep learning: A configuration-first framework with three-tier verification. *arXiv preprint arXiv:2509.06977*, 2025.

NVIDIA. Determinism in deep learning. NVIDIA developer documentation / framework-determinism.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22, 2021.

PyTorch Team. Reproducibility — pytorch documentation. https://pytorch.org/docs/stable/notes/randomness.html.

Sanjif Shanmugavelu, Mathieu Taillefumier, Christopher Culver, Oscar Hernandez, Mark Coletti, and Ada Sedova. Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications. *arXiv preprint arXiv:2408.05148*, 2024.

Megha Srivastava, Simran Arora, and Dan Boneh. Optimistic verifiable training by controlling hardware nondeterminism. *arXiv preprint arXiv:2403.09603*, 2024.

Cecilia Summers and Michael J. Dinneen. Nondeterminism and instability in neural network optimization. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501*, 2025.

Donglin Zhuang, Xingyao Zhang, Shuaiwen Leon Song, and Sara Hooker. Randomness in neural network training: Characterizing the impact of tooling. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022.